

happens. Agents subscribe to the events relevant to their function and act when those events arrive. The result is real-time awareness without the overhead of continuous polling, and asynchronous coordination without the brittleness of synchronous dependency chains.

Events address each of the integration problems outlined above. Real-time triggers tell an agent precisely when to act, rather than requiring it to guess through polling. Multiple agents can respond to the same event simultaneously, turning sequential processing into parallel coordination. Events can be persisted and replayed, so a temporarily unavailable system does not cause lost coordination. And new agents can join an ecosystem simply by subscribing to relevant event streams, without modifying existing integrations.

It is worth being explicit about one thing: EDA does not replace APIs. An event-driven agent still needs API access to retrieve data, execute transactions, and interact with external systems. The two patterns are complementary. APIs provide the ‘what’ — access to data and functionality. Events provide the ‘when’ — real-time triggers and coordination signals. Production agentic systems need both.

What production architecture requires

The gap between an agentic proof-of-concept and a production agentic system is largely an architectural gap. The components that close it share four characteristics.

An asynchronous communication backbone: An event streaming platform that decouples producers from consumers, provides reliable ordered message delivery, and handles speed mismatches between systems. This is what allows agents to scale independently without re-engineering the integration layer every time a new agent is added.

Intelligent orchestration: A layer that maintains awareness of which

agents exist, what they are capable of, and which tasks they are authorised to handle. This decomposes incoming requests into discrete tasks, routes each to the appropriate agent, and manages sequencing across complex workflows. Emerging standards such as the Model Context Protocol (MCP) and the Agent-to-Agent Protocol (A2A) are beginning to formalise how agents connect to tools, data sources, and each other — a development Gartner predicts will push 40% of enterprise applications to include task-specific AI agents by end of 2026, up from less than 5% in 2025.

Selective data handling: One of the less obvious production principles is minimising what gets passed to language models. Pushing entire datasets into an LLM is expensive, slow, and increases hallucination risk. Well-designed systems perform retrieval, filtering, and transformation outside the model, sending only what is necessary for the reasoning task.

Observability from day one: Autonomous agents operating across critical business processes require the same infrastructure as any distributed system: structured logging, distributed tracing, retry logic, and failure handling. An unobserved agent failure in a payment workflow is not a monitoring gap. It is a business risk. Notably, Deloitte’s 2025 survey found that 93% of AI budgets go towards technology while only 7% fund training and governance readiness — a ratio that helps explain why observability is consistently the last capability enterprises build and the first one they regret not having.

A production AI system looks less like a chatbot, more like microservices.

A practical sequence for getting started

Architectural transformation does not happen in a single project. But there is a sequence that enterprise teams have found effective.

- Audit your current integration architecture. Identify where

synchronous bottlenecks create real business risk. This surfaces the specific failure modes that event-driven patterns will address.

- Identify two or three high-value processes that are currently slow because they wait for approvals, data availability, and system responses. Real-time event awareness produces immediately measurable improvement here.
- Build the event-driven foundation before you build the agents. Most teams start with the AI. But agents built on polling-and-API architectures hit the same walls that stalled earlier initiatives. The foundation must come first.
- Design agents with narrow scope, clear input/output contracts, and explicit failure handling. Agents that do too much fail in opaque ways. The microservices discipline that has served distributed systems teams well applies directly here.
- Invest in observability before you need it. The enterprises scaling agentic AI successfully are not those with the most sophisticated models. They are those with the clearest visibility into what their agents are doing and why.

The stakes are higher than they appear

Gartner predicts that over 40% of agentic AI projects will be cancelled by the end of 2027 — not because the technology is immature, but because of escalating costs, unclear business value, and inadequate risk controls. That is a striking forecast for a technology enterprises are rushing to adopt.

Deloitte’s TMT 2025 Predictions report estimated that 25% of genAI users would launch agentic pilots or proofs-of-concept in 2025 growing to 50% by 2027. Yet Deloitte’s own Tech Trends 2026 study, published just months later, found that only 11% of organisations have agents running in production. The gap between those two numbers is the integration problem in statistical form.